

Práctica final - Servicio de envío de mensajes



Sistemas distribuidos

Curso 2025/26

Nombre, apellidos y NIA	
Alumna 1:	Ana Grima Vázquez de Prada (100495785) 100495785@alumnos.uc3m.es Grupo 80 - Grado en Ingeniería Informática
Alumna 2:	Alicia Mei Garcia Morin (100498828) 100498828@alumnos.uc3m.es Grupo 80 - Doble grado en Ingeniería Informática y ADE

Índice

Declaración de uso de IA.....	2
1. Introducción.....	3
2. Descripción general del sistema.....	3
3. Diseño realizado.....	4
3.1. Arquitectura cliente-servidor.....	4
3.2. Diseño del servidor en C.....	5
3.3. Diseño del cliente en Python.....	5
3.4. Protocolo de comunicación.....	6
3.5. Almacenamiento y gestión de mensajes pendientes.....	7
3.6. Envío de ficheros adjuntos.....	7
3.7. Servicio web conversor de mensajes.....	8
3.8. Servicio RPC de registro.....	8
4. Descripción del código.....	9
4.1. Módulo server.c.....	9
4.2. Módulo net_utils.c.....	9
4.3. Módulo user_store.c.....	9
4.4. Módulo operations.c.....	9
4.5. Módulo delivery.c.....	10
4.6. Cliente Python (client.py).....	10
5. Compilación y ejecución.....	10
6. Plan de pruebas y resultados.....	11
7. Conclusiones y problemas encontrados.....	15

Declaración de uso de IA

Durante el desarrollo de esta práctica, se han utilizado herramientas de IA como herramientas de apoyo (GitHub Copilot y ChatGPT) para depuración de errores, resolución de dudas puntuales, generación de código repetitivo y revisión del contenido textual de la memoria.

Estas herramientas se han utilizado únicamente como soporte al proceso de aprendizaje y desarrollo, manteniéndose en todo momento la comprensión, diseño e implementación original del sistema por nuestra parte.

1. Introducción

El objetivo de esta práctica ha sido diseñar e implementar un sistema distribuido de mensajería entre usuarios utilizando distintas tecnologías vistas a lo largo de la asignatura de Sistemas Distribuidos.

El sistema desarrollado permite registrar usuarios, establecer conexiones entre clientes, enviar y recibir mensajes de texto, transferir ficheros adjuntos y consultar los usuarios conectados en cada momento. Además, se han incorporado dos componentes adicionales: un servicio web encargado de normalizar los mensajes antes de enviarlos y un servicio RPC destinado al registro de operaciones realizadas en el sistema.

Durante el desarrollo de la práctica se han aplicado distintos conceptos trabajados en la asignatura, como la comunicación mediante sockets TCP, la concurrencia basada en hilos, la sincronización mediante mutex, las llamadas a procedimientos remotos (RPC) y la interacción entre servicios distribuidos.

En esta memoria se describe el diseño realizado, la organización del código, el protocolo de comunicación utilizado y las principales decisiones adoptadas durante la implementación, así como las pruebas llevadas a cabo para verificar el correcto funcionamiento del sistema.

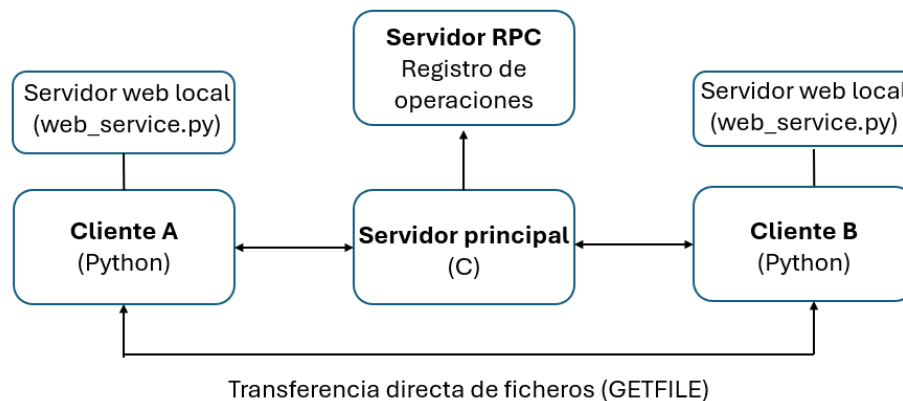
2. Descripción general del sistema

El sistema desarrollado está compuesto por cuatro componentes principales que colaboran entre sí para ofrecer un servicio distribuido de mensajería entre usuarios:

- Servidor principal (*server*): implementado en lenguaje C. Gestiona el registro y conexión de usuarios, el envío de mensajes y el almacenamiento de mensajes pendientes.
- Servicio web (*web_service.py*): servicio desarrollado con Flask que normaliza los mensajes antes de enviarlos, eliminando espacios redundantes.
- Servidor RPC (*logger_server*): servicio ONC-RPC encargado de registrar las operaciones realizadas por los usuarios.
- Cliente (*client.py*): implementado en Python. Proporciona una interfaz interactiva desde la que los usuarios pueden ejecutar las distintas operaciones del sistema y recibir mensajes de manera concurrente mediante un hilo receptor.

La arquitectura principal sigue un modelo cliente-servidor. Los clientes se comunican con el servidor principal mediante sockets TCP para realizar operaciones como REGISTER, CONNECT, SEND o USERS. Además, cada cliente mantiene un hilo receptor independiente que permanece escuchando conexiones entrantes para recibir mensajes o atender solicitudes de transferencia de ficheros.

La siguiente figura muestra la arquitectura general del sistema:



El flujo general de funcionamiento es el siguiente: un usuario inicia el cliente, se registra en el servidor y se conecta indicando el puerto en el que su hilo receptor quedará escuchando. A partir de ese momento puede enviar mensajes a otros usuarios conectados o desconectados.

Si el destinatario no se encuentra conectado, el servidor almacena temporalmente el mensaje y lo entrega automáticamente cuando el usuario vuelve a conectarse. En el caso de los ficheros adjuntos, el servidor únicamente coordina la operación inicial y la transferencia del contenido se realiza directamente entre clientes mediante la operación GETFILE.

3. Diseño realizado

3.1. Arquitectura cliente-servidor

El sistema sigue una arquitectura cliente-servidor. El servidor principal actúa como punto central de coordinación y mantiene la información global del sistema: usuarios registrados, estado de conexión y mensajes pendientes de entrega.

Los clientes se comunican con el servidor mediante sockets TCP para realizar operaciones como REGISTER, CONNECT, SEND o USERS. Cada operación establece una nueva conexión TCP con el servidor, realiza el intercambio de mensajes correspondiente y cierra la conexión al finalizar.

Una vez conectado, cada cliente crea además un hilo receptor independiente que permanece escuchando conexiones entrantes. Este hilo permite recibir mensajes enviados por el servidor y atender solicitudes de transferencia de ficheros de otros clientes.

Aunque la arquitectura principal es cliente-servidor, la operación GETFILE sigue un modelo peer-to-peer: el cliente que solicita el fichero se conecta directamente al hilo receptor del cliente propietario del archivo, sin que el servidor intervenga en la transferencia del contenido binario.

3.2. Diseño del servidor en C

El servidor principal (*server*) está implementado en lenguaje C y sigue un modelo concurrente multihilo. Su función es gestionar el estado global del sistema: usuarios registrados, conexiones activas y mensajes pendientes de entrega.

La implementación se ha dividido en varios módulos independientes para separar responsabilidades y facilitar el mantenimiento del código:

Fichero	Responsabilidad
<i>server.c</i>	Punto de entrada del servidor. Inicializa el socket TCP de escucha y crea un hilo por cada conexión entrante.
<i>server_common.h</i>	Declaraciones compartidas, constantes y estructuras de datos principales.
<i>net_utils.c</i>	Funciones auxiliares de comunicación TCP y envío/recepción segura de datos.
<i>user_store.c</i>	Gestión de usuarios y almacenamiento de mensajes pendientes.
<i>operations.c</i>	Implementación de las operaciones del protocolo
<i>delivery.c</i>	Entrega de mensajes y notificaciones entre clientes.
<i>rpc_logger.c</i>	Comunicación con el servidor RPC de registro de operaciones.

La estructura principal del servidor es una lista enlazada de usuarios (*user_t*), protegida mediante un mutex global (*g_users_mutex*) para evitar condiciones de carrera entre hilos concurrentes. Cada usuario almacena su nombre, estado de conexión, dirección IP, puerto de escucha y cola de mensajes pendientes.

El modelo de concurrencia utilizado sigue el patrón *thread-per-connection*: el hilo principal permanece bloqueado en *accept()* esperando nuevas conexiones y, cuando llega una petición, crea un nuevo hilo mediante *pthread_create()* para atenderla de forma independiente. Los hilos se crean en modo detached, para no tener que hacer *join()*.

Cuando el servidor recibe una señal *SIGINT* (*Ctrl+C*), cierra el socket de escucha y libera toda la memoria dinámica utilizada por la lista de usuarios y mensajes pendientes, permitiendo una finalización limpia del sistema.

3.3. Diseño del cliente en Python

El cliente (*client.py*) está implementado en Python y proporciona una interfaz interactiva desde la que el usuario puede ejecutar todas las operaciones disponibles del sistema. El diseño del cliente utiliza dos hilos de ejecución concurrentes:

- Un hilo principal encargado de leer y procesar los comandos introducidos por el usuario.
- Un hilo receptor (*listener*) que permanece escuchando conexiones entrantes para recibir mensajes del servidor y atender solicitudes GETFILE de otros clientes.

Cuando el usuario ejecuta la operación CONNECT, el cliente crea un socket de escucha en un puerto libre asignado automáticamente por el SO y lanza el hilo receptor asociado a dicho puerto. La dirección IP y el puerto de escucha se envían al servidor para que este pueda entregar mensajes posteriormente.

Cada operación (REGISTER, SEND, USERS, ...) establece una nueva conexión TCP con el servidor, realiza el intercambio de mensajes correspondiente y cierra la conexión al finalizar. Además, el cliente mantiene una caché local con la información de usuarios conectados (nombre, IP, puerto), utilizada durante la transferencia directa de ficheros (GETFILE).

Para evitar problemas de concurrencia en la salida por pantalla, las operaciones de impresión se protegen mediante un objeto *threading.Lock*, evitando que los mensajes del hilo receptor y del hilo principal aparezcan mezclados en consola.

3.4. Protocolo de comunicación

Cada operación establece una nueva conexión TCP, realiza el intercambio de mensajes correspondiente y cierra la conexión al finalizar. Nuestro protocolo utiliza cadenas terminadas en '\0' para el envío de datos y códigos de resultado codificados como un único byte. Además de las peticiones cliente-servidor, el sistema incluye conexiones iniciadas por el servidor hacia los clientes para entregar mensajes y confirmaciones de entrega.

Operación	Secuencia de mensajes cliente a servidor	Secuencia de mensajes servidor a cliente
REGISTER	Cadena "REGISTER", nombre de usuario.	Byte resultado (0=OK, 1=ya existe, 2=error).
UNREGISTER	"UNREGISTER", nombre.	Byte (0=OK, 1=no existe, 2=error).
CONNECT	"CONNECT", nombre, puerto de escucha del cliente.	Byte (0=OK, 1=no existe, 2=ya conectado, 3=error).
DISCONNECT	"DISCONNECT", nombre.	Byte (0=OK, 1=no existe, 2=no conectado, 3=error).
SEND	"SEND", remitente, destinatario, mensaje.	Byte resultado + identificador si OK.
SENDATTACH	"SENDATTACH", remitente, destinatario, mensaje, nombre de fichero.	Byte resultado + identificador si OK.
USERS	"USERS", nombre del solicitante.	Byte resultado + lista de usuarios conectados.
SEND MESSAGE		Al hilo receptor: "SEND MESSAGE", remitente, identificador, texto del mensaje.
SEND MESSAGE ATTACH		Al hilo receptor: "SEND MESSAGE ATTACH", remitente, identificador, mensaje

		y nombre de fichero.
SEND MESS ACK		Al hilo receptor del remitente: "SEND MESS ACK", identificador del mensaje entregado.
SEND MESS ATTACH ACK		Al hilo receptor del remitente: "SEND MESS ATTACH ACK", identificador del mensaje entregado.
GETFILE	Cliente → cliente: nombre solicitante y fichero Transferencia binaria del fichero	

3.5. Almacenamiento y gestión de mensajes pendientes

Cuando el destinatario de un mensaje no está conectado en el momento del envío, el mensaje se encola en la lista de mensajes pendientes asociada a dicho usuario. Cada usuario dispone de una cola FIFO implementada con una lista enlazada simple: el servidor mantiene punteros al primer y al último elemento para que la inserción sea $O(1)$.

Cada mensaje pendiente almacena su identificador numérico, el nombre del remitente, el texto del mensaje y, en el caso de SENDATTACH, información sobre el fichero adjunto asociado. El identificador se incrementa por remitente y, en caso de desbordamiento del tipo unsigned int, vuelve al valor 1.

Cuando un usuario se conecta, el servidor intenta entregar todos sus mensajes pendientes uno a uno. Si la entrega falla (por ejemplo, el cliente no acepta la conexión), el servidor marca al cliente como desconectado y deja de intentar la entrega. Cada mensaje entregado con éxito se elimina inmediatamente de la cola del servidor: el servidor solo almacena mensajes que aún no han sido recibidos.

Al eliminar un usuario mediante UNREGISTER, el servidor libera también toda la memoria asociada a su cola de mensajes pendientes.

3.6. Envío de ficheros adjuntos

La operación SENDATTACH sigue un funcionamiento similar al envío de mensajes normales descrito anteriormente, incorporando además la gestión de ficheros adjuntos. La transferencia de ficheros entre clientes se realiza de forma directa, sin que el servidor principal intervenga en el envío del contenido binario. El funcionamiento se divide en dos fases:

1. **Notificación del fichero:** cuando un usuario ejecuta SENDATTACH, el servidor almacena el mensaje junto con el nombre del fichero asociado y lo entrega al destinatario mediante la operación SEND MESSAGE ATTACH. El destinatario recibe el mensaje y visualiza el nombre del fichero adjunto.
2. **Transferencia del contenido:** el destinatario puede solicitar posteriormente el fichero mediante GETFILE. Para ello, el cliente obtiene la IP y el puerto del usuario propietario a través de la operación USERS y establece una conexión TCP directa con su hilo receptor. El cliente propietario abre el fichero solicitado y envía primero su tamaño y posteriormente el contenido binario en bloques.

Este diseño evita que el servidor tenga que almacenar o transmitir archivos binarios, manteniendo su función principal como coordinador del sistema de mensajería.

3.7. Servicio web conversor de mensajes

El servicio web (*web_service.py*) se ha implementado en Python utilizando Flask. Su función es normalizar los mensajes antes de que el cliente los envíe al servidor principal.

El servicio expone el endpoint *POST /normalize*, que recibe un objeto JSON con el campo *text* y devuelve otro JSON con el campo *normalized_text*. La normalización consiste en eliminar espacios en blanco repetidos mediante `" ".join(text.split())`, de forma que las palabras queden separadas por un único espacio.

Cada cliente invoca este servicio antes de ejecutar las operaciones SEND y SENDATTACH. El servicio escucha en el puerto 5000 y se ejecuta de forma independiente al servidor principal, por lo que su caída no afecta al funcionamiento básico del sistema de mensajería.

3.8. Servicio RPC de registro

El servicio de registro se ha implementado mediante ONC-RPC utilizando *rpcgen*. La interfaz se define en el fichero *logger.x*, donde se declara la estructura *log_request*, formada por tres cadenas: nombre de usuario, operación realizada y nombre de fichero en caso de SENDATTACH.

El servidor principal (*server*) actúa como cliente RPC. Cada vez que procesa una operación de usuario, invoca al servidor RPC (*logger_server*) para registrar la acción realizada. La dirección del servidor RPC se obtiene mediante la variable de entorno *LOG_RPC_IP*.

El procedimiento remoto utilizado es *LOG_OPERATION*, que recibe una estructura *log_request* y devuelve un entero como código de resultado. En el servidor RPC, la operación se muestra por pantalla, incluyendo el nombre del fichero cuando se trata de una operación SENDATTACH.

Si *LOG_RPC_IP* no está definida o el servicio RPC no está disponible, el servidor principal continúa funcionando sin registrar operaciones, evitando que un fallo del servicio auxiliar bloquee el sistema de mensajería.

4. Descripción del código

En este apartado se resumen las principales funciones implementadas en cada módulo, sin reproducir el código fuente.

4.1. Módulo `server.c`

- *main()*: inicializa el servidor, crea el socket de escucha y entra en el bucle principal de aceptación de conexiones.
- *client_thread_main()*: función ejecutada por cada hilo de atención, encargada de procesar una petición de cliente.
- *handle_sigint()*: permite finalizar el servidor limpiamente liberando recursos y desbloqueando *accept()*.

4.2. Módulo `net_utils.c`

- *send_all()* / *recv_all()*: garantizan el envío y recepción completos de datos TCP, incluso ante envíos parciales.
- *send_cstring()* / *recv_cstring()*: funciones auxiliares para transmitir cadenas terminadas en `'\0'`.
- *connect_to_host()*: establece conexiones TCP salientes con clientes o servicios auxiliares.

Estas funciones gestionan correctamente envíos parciales e interrupciones EINTR, garantizando transmisiones TCP robustas.

4.3. Módulo `user_store.c`

- *register_user_internal()* / *unregister_user_internal()*: altas y bajas de usuarios.
- *find_user_locked()*: búsqueda de usuarios protegida mediante mutex.
- *add_pending_message_locked()* / *remove_pending_message_locked()*: gestión de la cola FIFO de mensajes pendientes.
- *next_message_id_for_sender()*: generación de identificadores de mensaje.

4.4. Módulo `operations.c`

- *handle_register_operation()* y *handle_connect_operation()*: gestión de operaciones de conexión.
- *handle_send_common()*: lógica compartida entre SEND y SENDATTACH.
- *handle_users_operation()*: construcción y envío de la lista de usuarios conectados.

4.5. Módulo `delivery.c`

- `deliver_message_to_endpoint()`: envío de mensajes al hilo receptor del cliente destinatario.
- `send_ack_to_sender_if_connected()`: envío de confirmaciones ACK al remitente.
- `deliver_pending_messages_for_user()`: entrega automática de mensajes pendientes tras CONNECT.

4.6. Cliente Python (`client.py`)

- `_listener_loop()`: hilo receptor encargado de aceptar conexiones entrantes.
- `_normalize_message()`: integración con el servicio web.
- `_send_file() / _recv_file()`: transferencia binaria de ficheros.
- `_refresh_users_cache()`: actualización de la caché de usuarios conectados.
- `shell()`: bucle principal interactivo del cliente.

5. Compilación y ejecución

Para compilar el proyecto, hay que situarse en el directorio raíz del proyecto y ejecutar:

```
make clean  
make
```

El Makefile utiliza `rpcgen` para generar automáticamente los ficheros derivados de la interfaz RPC definida en `logger.x` (`logger_clnt.c`, `logger_svc.c`, `logger_xdr.c` y `logger.h`).

La compilación se realiza mediante `gcc` utilizando los flags `-Wall -Wextra -pedantic -std=c11 -pthread`, por lo que el proyecto compila sin warnings. Los ejecutables generados son:

- `server`: servidor principal de mensajería.
- `logger_server`: servidor RPC de registro de operaciones.

Se recomienda arrancar los distintos componentes en el siguiente orden:

Servicio web (en cada máquina cliente): `python3 web_service.py`

Servidor RPC: `./logger_server`

Servidor principal: `export LOG_RPC_IP=<IP_RPC> ./server -p <PUERTO>`

Cientes: `python3 client.py -s <IP_SERVIDOR> -p <PUERTO>`

Para cerrar el servidor principal y liberar todos los recursos: `Ctrl+C`

Si la variable de entorno `LOG_RPC_IP` no está definida, el sistema continúa funcionando sin el servicio RPC de registro. De forma análoga, si el servicio web no está disponible, el cliente envía los mensajes sin normalización previa.

Para que la transferencia directa de ficheros funcione correctamente en máquinas o contenedores distintos, los clientes deben ser accesibles mediante la IP y el puerto publicados durante la operación `CONNECT` y obtenidos posteriormente mediante `USERS`.

Las pruebas del sistema las hicimos sobre las máquinas `guernika.lab.inf.uc3m.es`, utilizando varias terminales SSH simultáneas para ejecutar de forma concurrente el servidor principal, el servidor RPC, los servicios web y múltiples clientes conectados. Este entorno permitió verificar el funcionamiento distribuido del sistema utilizando direcciones IP distintas entre procesos.

6. Plan de pruebas y resultados

Las pruebas se organizaron por funcionalidades, cubriendo tanto flujos correctos como casos de error. En todos los casos se comprobó la salida del cliente y, cuando procedía, los mensajes mostrados por el servidor principal y por el servidor RPC.

ID	Descripción de la prueba	Resultados obtenidos
1	Se registran correctamente dos usuarios distintos (ana y alicia) desde clientes diferentes mediante la operación <code>REGISTER</code> . Además, se comprueba que el servidor principal procesa ambas peticiones y que el servidor RPC registra las operaciones realizadas.	Ambos usuarios fueron registrados correctamente. Los clientes recibieron <code>REGISTER OK</code> , el servidor principal mostró el alta de ambos usuarios y el servidor RPC registró las operaciones <code>REGISTER</code> correspondientes.
2	Se intenta registrar nuevamente un usuario ya existente (ana) para comprobar que el sistema detecta correctamente nombres duplicados y evita registros repetidos.	El sistema rechazó el registro duplicado correctamente. El cliente recibió <code>USERNAME IN USE</code> y el servidor principal mostró el fallo de la operación <code>REGISTER</code> .
3	Se conectan correctamente dos usuarios previamente registrados (ana y alicia) mediante la operación <code>CONNECT</code> . La prueba verifica la creación del hilo receptor del cliente, la actualización del estado de conexión en el servidor y el registro de la operación en el servicio RPC.	Ambos usuarios se conectaron correctamente. Los clientes recibieron <code>CONNECT OK</code> , el servidor principal mostró la conexión de ambos usuarios y el servidor RPC registró las operaciones <code>CONNECT</code> correspondientes.
4	Se intenta conectar un usuario no registrado (luis) para comprobar que el servidor valida correctamente la existencia previa del usuario antes de permitir la conexión.	El sistema rechazó correctamente la conexión. El cliente recibió <code>CONNECT FAIL, USER DOES NOT EXIST</code> y el servidor principal mostró el fallo de la operación <code>CONNECT</code> .
5	Se intenta conectar nuevamente un usuario que ya se encuentra conectado (ana) para comprobar que el servidor detecta conexiones duplicadas y mantiene un único estado activo por usuario.	El sistema rechazó correctamente la segunda conexión. El cliente recibió <code>USER ALREADY CONNECTED</code> , verificando que no se permiten conexiones duplicadas para un

		mismo usuario.
6	Se ejecuta la operación USERS desde un cliente conectado para comprobar que el servidor devuelve correctamente la lista de usuarios activos junto con su dirección IP y puerto de escucha. Además, se verifica el registro de la operación en el servidor RPC.	La operación se realizó correctamente. El cliente recibió la lista de usuarios conectados con sus respectivas IP y puertos, el servidor principal confirmó la petición USERS y el servidor RPC registró correctamente la operación realizada por ana.
7	Se ejecuta la operación USERS desde un cliente no conectado para comprobar que el sistema valida correctamente el estado de conexión antes de permitir consultar la lista de usuarios activos.	El sistema rechazó correctamente la operación. El cliente recibió CONNECTED USERS FAIL, USER IS NOT CONNECTED, verificando que solo los usuarios conectados pueden consultar la lista de usuarios activos.
8	Se envía un mensaje mediante la operación SEND entre dos usuarios conectados (alicia → ana) para comprobar el flujo completo de entrega inmediata, incluyendo la confirmación ACK al remitente, la recepción del mensaje por el destinatario y el registro de la operación en el servicio RPC y en el servicio web de normalización.	El mensaje fue entregado correctamente. El remitente recibió SEND OK - MESSAGE 1 y posteriormente SEND MESSAGE 1 OK, mientras que el destinatario recibió el mensaje con el formato esperado. El servidor principal mostró la entrega del mensaje, el servidor RPC registró la operación SEND y el servicio web recibió correctamente la petición POST /normalize.
9	Se envía un mensaje con múltiples espacios consecutivos para comprobar que el servicio web de normalización procesa correctamente el texto antes del envío al servidor principal. Además, se verifica que el servicio Flask recibe la petición HTTP correspondiente.	El mensaje fue enviado y entregado correctamente. El destinatario recibió el texto normalizado con un único espacio entre palabras, confirmando el correcto funcionamiento del servicio web. En la terminal del servicio Flask se registró correctamente la petición POST /normalize con código HTTP 200.
10	Se intenta enviar un mensaje mediante la operación SEND a un usuario inexistente (pepe) para comprobar que el servidor valida correctamente la existencia del destinatario antes de procesar el envío.	El sistema rechazó correctamente la operación. El cliente recibió SEND FAIL, USER DOES NOT EXIST y el servidor principal mostró el fallo correspondiente de la operación SEND. Además, el servicio web registró correctamente la petición POST /normalize asociada al mensaje enviado.
11	Se comprueba el almacenamiento y entrega automática de mensajes pendientes. Para ello, un usuario conectado (alicia) envía un mensaje a otro usuario desconectado (ana). Posteriormente, ana vuelve a conectarse y el servidor intenta entregar automáticamente el mensaje almacenado. También se verifica el envío del ACK al remitente y la intervención del servicio web de normalización.	El sistema almacenó correctamente el mensaje mientras ana estaba desconectada. Tras volver a ejecutar CONNECT, el destinatario recibió automáticamente el mensaje pendiente y el remitente recibió posteriormente la confirmación SEND MESSAGE 3 OK. El servidor principal mostró el almacenamiento y posterior entrega del mensaje, y el servicio web registró correctamente la petición POST /normalize.
12	Se intenta ejecutar dos veces consecutivas la	La primera desconexión se realizó

	operación DISCONNECT sobre el mismo usuario (ana) para comprobar que el servidor detecta correctamente cuando un usuario ya no se encuentra conectado.	correctamente y el cliente recibió DISCONNECT OK. Al intentar desconectar nuevamente al mismo usuario, el sistema rechazó la operación mostrando DISCONNECT FAIL, USER NOT CONNECTED, mientras que el servidor principal registró el fallo correspondiente.
13	Se intenta eliminar mediante UNREGISTER un usuario inexistente (luis) para comprobar que el servidor valida correctamente la existencia previa del usuario antes de procesar la baja.	El sistema rechazó correctamente la operación. El cliente recibió USER DOES NOT EXIST y el servidor principal mostró el fallo correspondiente de la operación UNREGISTER.
14	Se elimina correctamente un usuario registrado (ana) mediante la operación UNREGISTER para comprobar que el servidor borra al usuario del sistema y libera la información asociada. También se verifica el registro de la operación en el servidor RPC.	El usuario fue eliminado correctamente. El cliente recibió UNREGISTER OK, el servidor principal confirmó la baja del usuario y el servidor RPC registró correctamente la operación UNREGISTER.
15	Se intenta conectar un usuario previamente eliminado mediante UNREGISTER para comprobar que la baja se ha realizado correctamente y que el usuario ya no existe en el sistema. Posteriormente, el usuario se vuelve a registrar y conectar para continuar con las siguientes pruebas.	El sistema rechazó correctamente la conexión del usuario eliminado mostrando CONNECT FAIL, USER DOES NOT EXIST. Tras volver a ejecutar REGISTER y CONNECT, el usuario pudo registrarse y conectarse nuevamente sin problemas.
16	Se envía un mensaje con fichero adjunto mediante la operación SENDATTACH entre dos usuarios conectados para comprobar el flujo completo de envío de adjuntos, incluyendo la entrega del mensaje, la notificación del nombre del fichero, el envío del ACK correspondiente, el registro de la operación en el servidor RPC y la intervención del servicio web de normalización.	El mensaje con adjunto fue enviado y entregado correctamente. El remitente recibió la confirmación de envío y posteriormente el ACK de entrega, mientras que el destinatario recibió el mensaje junto con el nombre del fichero adjunto. El servidor principal registró correctamente la operación, el servidor RPC mostró la operación SENDATTACH junto con la ruta del fichero y el servicio web registró correctamente la petición POST /normalize.
17	Se realiza una transferencia directa de fichero mediante la operación GETFILE para comprobar que el contenido del archivo se transmite correctamente entre clientes sin pasar por el servidor principal. Además, se verifica previamente la obtención de la IP y el puerto del cliente propietario mediante la operación USERS.	La transferencia se realizó correctamente. El cliente solicitante recibió FILE TRANSFER OK y el fichero descargado contenía exactamente el contenido esperado. La operación confirmó el correcto funcionamiento del modelo peer-to-peer utilizado para la transferencia de archivos entre clientes.
18	Se intenta realizar una transferencia mediante GETFILE hacia un usuario desconectado para comprobar que el cliente detecta correctamente que el propietario del fichero no está disponible y rechaza la operación.	El sistema detectó correctamente que el usuario propietario del fichero no estaba conectado. El cliente mostró FILE TRANSFER FAILED, user not connected, verificando el manejo correcto de errores en

		la transferencia directa de archivos.
19	Se intenta enviar un fichero adjunto inexistente mediante la operación SENDATTACH para comprobar que el cliente valida correctamente la existencia del archivo antes de iniciar el envío.	El cliente rechazó correctamente la operación mostrando SENDATTACH FAIL, verificando que no es posible enviar ficheros inexistentes como adjuntos.
20	Se intenta enviar un fichero adjunto utilizando una ruta relativa mediante SENDATTACH para comprobar que el cliente exige correctamente rutas absolutas antes de permitir la operación.	El cliente rechazó correctamente la operación mostrando SENDATTACH FAIL, verificando que el sistema únicamente permite enviar ficheros especificados mediante rutas absolutas.
21	Se intenta enviar un mensaje cuyo tamaño supera el límite permitido de 255 bytes para comprobar que el cliente valida correctamente la longitud máxima antes de enviar la petición al servidor.	El cliente rechazó correctamente la operación mostrando SEND FAIL, verificando que el sistema controla adecuadamente el tamaño máximo permitido para los mensajes.
22	Se ejecutan varias operaciones consecutivas (USERS, SEND y SENDATTACH) para comprobar el correcto funcionamiento del servicio RPC de registro. La prueba verifica que el servidor RPC recibe y muestra correctamente el usuario y la operación realizada, incluyendo además la ruta del fichero en el caso de SENDATTACH, sin registrar el contenido de los mensajes.	El servidor RPC registró correctamente todas las operaciones realizadas. Para operaciones normales se mostraron el usuario y la operación correspondiente, mientras que en SENDATTACH también se registró correctamente la ruta del fichero adjunto. No se mostró el contenido de los mensajes, verificando el comportamiento esperado del servicio RPC.
23	Se realizan múltiples envíos de mensajes simultáneos entre varios clientes conectados para comprobar el comportamiento concurrente del servidor y verificar que el modelo multihilo permite atender varias operaciones de forma paralela sin bloqueos ni inconsistencias.	El servidor gestionó correctamente múltiples operaciones concurrentes. Ambos clientes enviaron y recibieron mensajes simultáneamente, el servidor continuó aceptando nuevas conexiones y no se produjeron bloqueos ni inconsistencias en la entrega de mensajes, verificando el correcto funcionamiento del modelo multihilo implementado.
24	Se ejecuta la operación QUIT desde distintos clientes para comprobar que el cierre del cliente se realiza correctamente y que, en caso de estar conectado, se ejecuta internamente la operación DISCONNECT antes de finalizar.	Los clientes finalizaron correctamente mostrando +++ FINISHED +++. Además, cuando el usuario se encontraba conectado, el cliente realizó correctamente la desconexión antes de cerrar la aplicación.
25	Se fuerza la finalización del servidor principal mediante Ctrl+C para comprobar que el manejador de SIGINT realiza correctamente el cierre del socket de escucha y la liberación de los recursos dinámicos utilizados por el sistema.	El servidor finalizó correctamente sin bloqueos ni errores, liberando la memoria asociada a usuarios y mensajes pendientes y cerrando adecuadamente el socket de escucha.

7. Conclusiones y problemas encontrados

Durante el desarrollo de la práctica hemos podido aplicar de forma práctica muchos de los conceptos vistos en la asignatura, especialmente la comunicación mediante sockets TCP, el diseño de protocolos de aplicación, la concurrencia con hilos, el uso de RPC y la integración de servicios web. A medida que la práctica fue creciendo, también fuimos entendiendo mejor la diferencia entre las distintas tecnologías de comunicación distribuidas y cuándo tiene sentido utilizar cada una.

Uno de los aspectos más importantes ha sido la definición correcta del protocolo de comunicación entre cliente y servidor. Aunque conceptualmente las operaciones eran sencillas, pequeños errores en el orden de envío de los campos, en los nombres exactos de las operaciones o en la gestión de cadenas terminadas en `\0` podían provocar que cliente y servidor dejaran de entenderse correctamente. Por ello, gran parte del trabajo ha consistido en depurar problemas de comunicación y asegurar que ambos extremos siguieran exactamente el mismo protocolo.

Otro punto especialmente relevante ha sido la concurrencia. El servidor debía ser capaz de atender varios clientes simultáneamente, lo que obligó a implementar un modelo multihilo y proteger correctamente las estructuras compartidas mediante mutex. También resultó importante gestionar correctamente situaciones de error, como clientes que se desconectan de forma inesperada o mensajes pendientes que deben almacenarse y entregarse posteriormente.

En la segunda parte de la práctica, la transferencia de ficheros añadió bastante complejidad al sistema. Uno de los requisitos principales era que el contenido de los archivos no pasara por el servidor principal, sino directamente entre clientes mediante sockets. Para ello fue necesario ampliar la operación `USERS` para incluir IP y puerto, además de coordinar correctamente las operaciones `SENDATTACH` y `GETFILE`.

La integración del servicio web y del servicio RPC también nos permitió trabajar con arquitecturas más modulares y comprender cómo distintos servicios pueden colaborar dentro de un mismo sistema distribuido. En particular, el uso de `rpcgen`, la configuración de `rpcbind` y la gestión de las llamadas RPC fueron partes que inicialmente resultaron complejas.

En general, la práctica nos ha parecido una carga de trabajo considerable, especialmente al coincidir con otros ejercicios y entregas de la asignatura. Gran parte del tiempo se ha dedicado a depurar errores de concurrencia y problemas de comunicación difíciles de detectar. Aun así, estamos satisfechas con el resultado final y con todo lo aprendido durante el desarrollo del proyecto. Consideramos que la práctica nos ha ayudado a entender mucho mejor cómo se construye realmente un sistema distribuido completo y cuáles son los problemas que aparecen en este tipo de aplicaciones.